

READING SHEET

Server & Client Components

Module 3 · Frontend: React & Next.js · Day 38 · Week 8

PROGRAM CodeOps · Full Stack	WORK TYPE Revision + mini-project	EST. TIME 2–3 hours
--	---	-------------------------------

Tip How to use this sheet

This is the idea sitting underneath the last two days. In the app router a component runs on the server unless you say otherwise, ships no JavaScript to the browser, and can talk to a database directly. This sheet covers the two kinds of component, where the boundary between them belongs, what may cross it, and how to fetch data on the server. It also finally explains why `error.js` needed a directive.

Part 1 · Two Kinds of Component

Both kinds are ordinary React components written in the same syntax. The difference is *where they execute*, and therefore what they are allowed to touch. Everything you built on Vite was a client component, because there was no server — you have not lost anything, you have gained a second option.

SERVER COMPONENT	CLIENT COMPONENT
The default — no directive needed	Marked with "use client"
Runs on the server only	Runs on the server, then the browser
Ships zero JavaScript	Ships its code to the browser
Can read files and databases	Can use state and event handlers
Cannot use hooks or handlers	Cannot read secrets or a database
Can be async	Cannot be async

What a server component gains

```

// app/menu/page.js – no hook, no effect, no waterfall
export default async function MenuPage() {
  const dishes = await db.dish.findMany(); // straight to the database
  return <DishList dishes={dishes} />;
}

// no useState, no useEffect, no loading flag, no /api in between

```

The three pieces of state from Day 29, the effect, the cleanup and the error branch all disappear — the component simply awaits its data, because it is already running where the data lives. An API key used here is never sent to the browser: that is not a convention, the code itself never leaves the server.

What a client component gains

CategoryBar.jsx – interactivity needs the browser

```
"use client";
import { useState } from "react";

export default function CategoryBar({ categories }) {
  const [selected, setSelected] = useState("All");
  return categories.map((c) => (
    <button key={c} onClick={() => setSelected(c)}>{c}</button>
  ));
}
```

State, event handlers, effects, refs, context and browser APIs all require a client component. Note that it still renders on the server first to produce the initial HTML, then runs again in the browser to attach the handlers — that second step is called **hydration**.

Warning The error you will meet first

"You are importing a component that needs useState. It only works in a Client Component." The question to ask is: does this component need to respond to the person using it, or remember something between renders? If not, leave it on the server and ship no JavaScript for it.

Part 2 · The "use client" Boundary

The directive does not mark one component. It marks an **entry point** into the client, and everything imported beneath it joins the client bundle — which is why where you put it matters more than whether you use it.

the directive spreads downwards

```
// CategoryBar.jsx
"use client";
import { formatPrice } from "../format"; // now client code
import DishCard from "../DishCard";      // now a client component

// DishCard has no directive of its own, but it was imported
// from a client component – so it is client too.
```

Putting the directive high in the tree quietly converts everything below it: the bundle grows, the server-rendering advantage disappears, and nothing warns you. You only need it *at* the boundary — repeating it on every child file is harmless but says nothing new.

Push the boundary down

wrong, then right

```
// X "use client" at the top of page.js
//   the heading, the description, the dish cards and every
//   library they use now ship to the browser

// ✓ page.js stays a server component
//   only CategoryBar.jsx has "use client"
```

Tip A habit worth forming

When you hit the useState error, do not add the directive to the file you are in. Ask which part of that file actually needs state, extract it into its own component, and put the directive there instead. Find the smallest piece that genuinely needs interactivity — usually a button, a filter bar or a form — and make only that a client component.

What cannot cross the boundary

CAN BE PASSED AS A PROP	CANNOT
Strings, numbers, booleans	Functions
Arrays and plain objects	Class instances
Dates, Maps, Sets	Anything with methods
JSX elements	A database connection

A server component runs somewhere else, so its props have to be turned into data and sent over the wire. A function cannot be serialised, which is why `<CategoryBar onSelect={handleSelect} />` fails from a server component while passing an array of category names is fine. The one exception is **server actions**, where the framework sends a reference rather than the code — that is Day 39.

Reading the error messages

MESSAGE	WHAT IT MEANS
"only works in a Client Component"	A hook or handler in server code
"Functions cannot be passed"	A callback prop crossing the boundary
"window is not defined"	A browser API in server code
"async/await is not yet supported"	An async client component
Hydration mismatch	Server and client rendered different HTML

Each names a specific boundary decision — read it, decide which side that code belongs on, and move the directive rather than the code. Hydration mismatches usually come from rendering new `Date()` or `Math.random()`: move anything time- or random-dependent into an effect.

Part 3 · Composition Patterns

A client component cannot **import** a server component — but it can **receive** one as children. That single rule is what lets an interactive shell wrap server-rendered content without dragging it onto the client.

```
import versus children

// X a client component importing a server one
"use client";
import DishList from "../DishList";    // becomes client

// ✓ page.js – still a server component
<Accordion>      {/* client: holds open state */}
  <DishList />    {/* server: stays on the server */}
</Accordion>
```

The server renders `DishList` first and passes the finished result into the slot. `Accordion` never imports it, never sees its code, and cannot pull it into the bundle. It does not have to be children — any prop that receives JSX behaves the same way.

The shape to aim for

```

                                                                    app/menu/page.js
export default async function MenuPage() {
  const dishes = await getDishes();           // server
  return (
    <FilterShell>                               {/* client */}
      <DishList dishes={dishes} />             {/* server */}
    </FilterShell>);
}
// FilterShell ships ~2 kB. DishList ships nothing.

```

The filtering interaction runs in the browser, the forty dish cards are plain HTML, and the data query never leaves the server — and nothing about the code looks unusual. This is the Day 27 composition lesson made load-bearing: a component that accepts children does not need to know what it wraps.

Providers are client components

```

                                                                    app/providers.jsx – the standard pattern
"use client";
export function Providers({ children }) {
  return <CartProvider>{children}</CartProvider>;
}

// app/layout.js stays a server component
<body><Providers>{children}</Providers></body>

```

A provider holds state, so it must be a client component. Wrapping it in its own file keeps the root layout on the server while still giving every page access to the cart — and because children is *passed in* rather than imported, every page inside the provider is still server-rendered.

Addis Eats, sorted

COMPONENT	RUNS ON	WHY
app/menu/page.js	Server	Fetches dishes; no interactivity of its own
DishList, DishCard	Server	Pure markup from data — ships no JavaScript
CategoryBar	Client	Holds the selected category, handles clicks
AddToCartButton	Client	onClick, and writes to the cart store
Providers	Client	Context requires state
app/layout.js	Server	Passes children in, imports nothing client

Part 4 · Data on the Server

Fetching in a server component removes the waterfall, the loading flag and the API endpoint in between. Server components may be **async** — client components may not.

```

                                                                    the whole data layer, in one component
export default async function DishPage({ params }) {
  const dish = await getDish(params.id);
  if (!dish) notFound();
  return <DishCard {...dish} />; // loading.js covers the await
}

```

Day 29 taught loading, error and empty as three pieces of state. Here `loading.js` covers the wait, `error.js` catches the throw, and `notFound()` handles empty — the framework holds the states for you.

Parallel, not sequential

```
// X sequential - 600ms + 400ms
const dish = await getDish(id);
const reviews = await getReviews(id);

// ✓ parallel - 600ms total
const [dish, reviews] = await Promise.all([
  getDish(id), getReviews(id)]);
```

two awaits, one wait

Server-side awaits still run one after another if you write them that way — the waterfall moved, it did not vanish. The difference is that the round trips are now between your server and your database rather than a phone in Addis and a server abroad. Two independent Suspense boundaries are often better than `Promise.all`, because the fast section appears first.

Where each kind of data belongs

FETCH ON THE SERVER	FETCH ON THE CLIENT
The menu and dish details	Live order status polling
Anything a crawler should see	Search-as-you-type results
Anything needing a secret key	Data that changes on interaction
The first paint of a page	Infinite scroll after the first page
Data shared by every visitor	Data specific to one open tab

Tip Most data is server data

Start on the server and move to the client only when the fetch is triggered by something the person does — a much shorter list than the habits of the last two weeks suggest. Day 39 builds the endpoints a client component would call, and Day 41 gives client fetching a proper library.

Assigned Reading

Required

1. [nextjs.org — Server Components](#) What / why: the default model, what runs where, and the benefits in order.
2. [nextjs.org — Composition patterns](#) What / why: passing server components into client ones, and where the boundary belongs.

Optional — go deeper

- [Client Components](#) — the directive, hydration, and when you genuinely need it.
- [React Server Components](#) — the underlying React feature, explained without the framework.

Exercises

Work these in order and run `npm run build` after each one.

1. Convert the menu page to an async server component that awaits its dishes.
2. Delete the useFetch hook from that route and remove its loading and error state.
3. Search the project for "use client" and list every file that has it.
4. Move each directive down to the smallest component that genuinely needs it.
5. Extract the cart provider into its own client Providers component.
6. Wrap the server DishList in a client FilterShell using children rather than an import.
7. Record the First Load JS for /menu before and after, and explain the difference.

Mini-Project · Sorting the Boundary

Tip What you build

Go through Addis Eats component by component and place each one deliberately. Data fetched in async server components, "use client" only on interactive leaves, the cart provider isolated in its own file, and a bundle you can prove is smaller than it was. The application should behave identically — the whole exercise is about what the browser is asked to download.

Requirements

- The menu and dish routes as async server components with no fetching hooks.
- Every "use client" sitting on the smallest component that needs it.
- A Providers client component wrapping children, with layout.js still on the server.
- At least one client shell receiving server-rendered content through children.
- No callback props passed from a server component to a client one.
- BOUNDARY.md listing every component, which side it runs on, and one line of justification.
- Before and after First Load JS figures for /menu recorded in the README.

What to submit

Your GitHub repository with the sorted components, BOUNDARY.md, and the two bundle measurements.

Check yourself

- How many files contain "use client"? Can you justify each one?
- Does the menu page still work with every fetching hook deleted?
- Is DishList shipped to the browser, and can you prove it either way?
- Did the First Load JS for /menu actually go down?
- Does layout.js import anything that carries the directive?
- Can you explain in one sentence why error.js needs "use client"?